Unit – III
Software Project Estimation and Scheduling

By: Yagnik Hathaliya

Major Learning Outcomes

- **Responsibility of software project Manager**
- **Metrics for Size Estimation**
  - Line of Code
  - Function Points
- **Project Estimation Techniques using COCOMO model**
- **Project Scheduling**
  - Gantt Chart
  - Flow Chart
  - Sprint Burn Down Chart for Agile Model
- **Risk Management**
  - Risk Identification
  - Risk Assessment
  - Risk Control

Responsibility of Software Project Manager

- Software Project Manager is a professional responsible for planning, executing, and overseeing software development projects.
- Their primary role is to ensure that the project is completed on time, within budget, and meets the specified quality standards.
- General Responsibility/Activities are:
  - Project Proposal Writing
  - Project Cost Estimation
  - Project Planning and Project Scheduling
  - Project Staffing or Team Leadership and Management
  - Project Monitoring and Control
  - Software Configuration Management
  - Risk Management
  - Stakeholder Communication (Interfacing with Clients or other)
  - Managerial Report Writing and Presentations
  - Quality Assurance
  - Delivery and Closure etc.

- The Skills Required in Project Manager are:
  - Leadership Skills
  - Communication Skills
  - Organizational Skills
  - Problem-Solving Skills
  - Risk Management
  - Technical Skills
  - Budgeting and Financial Management
  - Negotiation Skills
  - Monitoring and Scheduling Capability
  - Critical Thinking and Decision Making
  - Team Building and Collaboration
  - Decision-Making Capabilities
  - Experience in the related area

Metrics for Project Size Estimation

- Project size estimation is critical for planning and managing software development projects, various metrics can be used to estimate project size, depending on the nature of the project and the information available.
- Metrics are the tools that help in better Monitoring and Control.
- Size of the program (or project) is neither the number of bytes that the source code occupies nor the byte size of the executable code, but it is an indicator of the effort and time required to develop the project
- Project Size is a measure of the problem complexity in terms of the effort and time required to develop the product.
- Two important Metrics to Estimate Size are:
  - Lines of Code (LOC)
  - Function Point (FP)

- **Lines of Code (LOC)**
  - In LOC, Project Size is estimated by counting the number of source instructions in the developed program, while counting the number of source instructions, lines used for commenting the code and the header lines should be ignored.
  - Determining the LOC count at the end of a project is a very simple job, However, accurate estimation of the LOC count at the beginning of a project is very difficult.
  - In order to estimate the LOC count at the beginning of a project, project managers usually divide the problem into modules, and each module into sub modules and so on, until the sizes of the different leaf-level modules can be approximately predicted.
  - Advantages:
    - Simplest Measure
    - Very Popular due to simplicity
  - Disadvantages:
    - Language Dependent
    - Gives only numerical values of problem size
    - Less Accuracy
    - Doesn't work well with non-procedural languages

– Example:

- You are estimating the size of a project that involves developing a basic web application with the following components:
    - User Authentication Module
    - Dashboard Module
    - Reporting Module
- Steps for LOC Estimation:
    - Breakdown of Components:
        - User Authentication Module:
            - Login functionality
            - Registration functionality
            - Password reset
        - Dashboard Module:
            - Display user-specific data
            - Navigation controls
        - Reporting Module:
            - Generate reports
            - Export data

- Estimate LOC for Each Functionality:
  - User Authentication Module:
    - Login: 200 LOC
    - Registration: 300 LOC
    - Password reset: 150 LOC
  - Dashboard Module:
    - Display data: 400 LOC
    - Navigation: 250 LOC
  - Reporting Module:
    - Generate reports: 500 LOC
    - Export data: 300 LOC
- Sum of Estimated LOC:
  - User Authentication Module: 200 + 300 + 150 = 650 LOC
  - Dashboard Module: 400 + 250 = 650 LOC
  - Reporting Module: 500 + 300 = 800 LOC
- Total Estimated LOC for the Project = 650 + 650 + 800 = 2100 LOC

- **Function Point (FP)**
  - It was first proposed by Albrecht in 1979.
  - It measures the functionality delivered to the user, independent of the programming language, tools, or development methodologies.
  - Function Points are especially useful during the early phases of a software project when detailed requirements are available but the design and code are not yet finalized.
  - FP considers five different component of the product to calculate the size, these are:
    - Number of External Inputs (EI): User-originated inputs that the system processes (e.g., forms, data entry screens).
    - Number of External Outputs (EO): Outputs generated by the system for the user (e.g., reports, results).
    - Number of External Queries (EQ): Input-output pairs where the system provides immediate results without complex logic (e.g., search functionality).
    - Number of Internal Logical Files (ILF): Logical data groups maintained within the system (e.g., database tables).
    - Number of External Interface Files (EIF): Data groups used by the system but maintained externally (e.g., external APIs or files).
  - Final Equation is : **FP = (0.65 * (EI + EQ + EO)) + (0.35 * (ILF + EIF))**

– Example: Library Management system

- External Inputs (EI):
  - User Searches for a Book: 3
  - Librarian Adds a Book: 4
  - Total EI = 7
- External Outputs (EO):
  - Book Details: 3
  - Total EO = 3
- External Inquires (EQ):
  - Book Availability: 3
  - Total EQ = 3
- Internal Logical Files (ILF):
  - Book Database: 10
  - Total ILF = 10
- External Interface Files (EIF):
  - Interface with External Book Catalog: 5
  - Total EIF = 5

FP = (0.65 * (EI + EQ + EO)) + (0.35 * (ILF + EIF)) = (0.65 * 13) + (0.35 * 15) = 13.7

- Advantages:
  - High Accuracy
  - Not Restricted to Code/Language
- Disadvantages:
  - It ignores quality of output
  - It doesn’t take algorithmic complexity into account.

Project Estimation Techniques using COCOMO Model

- Project estimation techniques are essential for planning and delivering projects on time and within budget, these methods help predict the resources, time, and cost required for a project.
- Project Estimation is a prediction or a rough idea to determine how much effort, time, cost or resource would take to complete a defined project task.
- There are three categories of estimation techniques:
  - Empirical Estimation Techniques (Based on Experience)
  - Heuristic Techniques (Based on research and Experiment)
  - Analytical Estimation Techniques (Based on Scientific Formula)
- Boehm guessed that any software development project can be classified into one of the following three categories based on the development complexity:
  - Organic
  - Semidetached
  - Embedded

- Organic:
  - A development project can be considered of organic type, if the project deals with developing a well understood application program, the size of the development team is small, and the team members are experienced in developing similar types of projects.
- Semidetached:
  - A development project can be considered of semidetached type, if the development consists of a mixture of experienced and inexperienced staff.
  - Team members may have limited experience on related systems but may be unfamiliar with some aspects of the system being developed.
- Embedded:
  - A development project is considered to be of embedded type, if the software being developed is strongly coupled to complex hardware.

- **COCOMO Model**
  - COCOMO (Constructive Cost Estimation Model) was proposed by Boehm in 1981, It provides estimates for software development effort, time, and cost based on the size of the software and other factors.
  - Three Stages of it: Each level achieves successive accuracy.
    - Basic COCOMO Model
    - Intermediate COCOMO Model
    - Complete COCOMO Model

- **Basic COCOMO Model**
  - It is a static and single valued model that provides a simple estimate of development effort and cost based on the size of the project in terms of Kilo Lines of Code (KLOC).
  - Suitable for small projects with well-understood requirements.
  - The basic COCOMO estimation model is given by the following expressions:

$$\text{Effort (E)} = a1 * (\text{KLOC})^{a2} \text{ PM}$$

$$\text{Tdev} = b1 * (\text{Effort})^{b2} \text{ Months}$$

  - Where KLOC is the estimated size of the software product expressed in Kilo Lines of Code.
  - a1, a2, b1, b2 are constants.
  - Tdev is the estimated time to develop the software, expressed in months.
  - Effort is the total effort required to develop the software product, expressed in person months (PMs).

- Estimation of Development Effort in Basic COCOMO Model

Organic	:	Effort (E) = $2.4\ (KLOC)^{1.05}$	PM
Semidetached	:	Effort (E) = $3.0\ (KLOC)^{1.12}$	PM
Embedded	:	Effort (E) = $3.6\ (KLOC)^{1.20}$	PM

- Estimation of Development Time in Basic COCOMO Model

Organic	:	$T_{dev} = 2.5\ (effort)^{0.38}$	Months
Semidetached	:	$T_{dev} = 2.5\ (effort)^{0.35}$	Months
Embedded	:	$T_{dev} = 2.5\ (effort)^{0.32}$	Months

- Cost Required to Develop the Product = Person Salary Per Month* Effort
- Example:
  - Assume that the size of an organic type software product has been estimated to be 2,000 lines of source code, determine the effort required to develop the software product and the nominal development time, if salary per person is 20000 then also calculate cost to develop the product.
  - Effort (E)= 2.4 * (2)^1.05 PM = 5 PM
  - Nominal Development Time T_{dev} = 2.5 * (5)^0.38 Months = 4.6 Months
  - Cost Required for Development the Product = 5 * 20000 = 100000 INR

- **Intermediate COCOMO Model**
  - Intermediate COCOMO model is an estimation technique used in software engineering to predict the cost, effort, and schedule required for software development, it improves upon the Basic COCOMO model by introducing cost drivers that account for various project, product, and personnel attributes.
  - The basic COCOMO model considers efforts and time development are the alone functions of product size, but some factors from other projects may also affect the efforts and time, therefore, in order to obtain an accurate estimation of the effort and project duration, the effect of all relevant parameters must be taken into account.
  - The intermediate COCOMO model doing this by using a set of 15 Cost Drivers (Multipliers) based on various attributes of software development.
  - The Intermediate COCOCMO model consumes these "Cost Drivers"
  - Project Manager doing the task of rating these 15 parameters in scale 1 to 3 and then cost driver values are estimated.

- The Cost Drivers can be grouped into four categories.
  - Product Attributes
    - Product Complexity (CPLX)
    - Required Software Reliability (RELY)
    - Database Size (DATA)
  - Computer Attributes
    - Execution Time Constraint (TIME)
    - Storage Constraint (STOR)
    - Virtual Machine Volatility (VIRT)
    - Turnaround Time (TURN)
  - Personal Attributes
    - Analyst Capability (ACAP)
    - Application Experience (AEXP)
    - Programmer Capability (PCAP)
    - Personnel Continuity (PCON)
    - Virtual Machine Experience (VEXP)
    - Language/Tool Experience (LEXP)
  - Project Attributes
    - Use of Software Tools (TOOL)
    - Development Schedule (SCED)

– Effort Estimation: The effort is estimated in person-months (PM) using the equation:

$$E = a \times (KLOC)^b \times \prod_{i=1}^{15} E_i$$

- **E**: Effort in person-months.
- **a, b**: Constants that depend on the software project type (organic, semi-detached, embedded).
- **KLOC**: Thousands of Lines of Code (size of the software).
- $\prod_{i=1}^{15} E_i$: Product of **effort multipliers** for 15 cost drivers.

– Development Time: The development time (T) is estimated in months:

$$T = c \times (E)^d$$

- **T**: Development time in months.
- **c, d**: Constants that depend on the project type.

- **Complete (Detail) COCOMO Model**
  - Complete COCOMO Model is the most advanced version of the COCOMO framework that extends the Intermediate COCOMO model by introducing phase-sensitive cost drivers and effort multipliers that allow detailed estimation at different stages of the software development lifecycle.
  - Major Shortcoming of both the Basic and Intermediate COCOMO models is that they consider a software product as a single similar entity, but most large systems are made up several smaller sub-systems and these subsystems may be differed in requirements, development complexities and may not have previous experience of similar developing.
  - Detailed COCOMO Model incorporates all characteristics of the intermediate version with an assessment of the cost driver's impact on each step (analysis, design, etc.) of the software engineering process, it estimates the effort and development time as the sum of the estimates for the individual subsystems.
  - The cost of each subsystem is estimated separately and summed up to give the overall cost of the system.

- The Complete COCOMO model divides the software lifecycle into distinct phases:
  - Requirements Analysis
  - Product Design
  - Detail Design
  - Coding and Unit Testing
  - Integration and System Testing

– Effort Estimation: Similar to the Intermediate model, the effort is calculated as:

$$E = a \times (KLOC)^b \times \prod_{i=1}^{15} E_i$$

- **E**: Effort in person-months.
- **a, b**: Constants that depend on the software project type (organic, semi-detached, embedded).
- **KLOC**: Thousands of Lines of Code (size of the software).
- $\prod_{i=1}^{15} E_i$: Product of **effort multipliers** for 15 cost drivers.

– Development Time: The development time (T) is estimated in months:

$$T = c \times (E)^d$$

- **T**: Development time in months.
- **c, d**: Constants that depend on the project type.

- Advantages:
  - Easy to Use
  - Structured and Systematic
  - Scalable
  - Phase-Wise Analysis
  - Identifies Key Factors Influencing Effort
  - Supports Decision-Making
- Disadvantages:
  - Reliance on Accurate Inputs
  - Not Suitable for Early Stages
  - Limited Applicability to Non-Software Projects
  - Ignores software safety issues
  - Ignores many hardware issues.

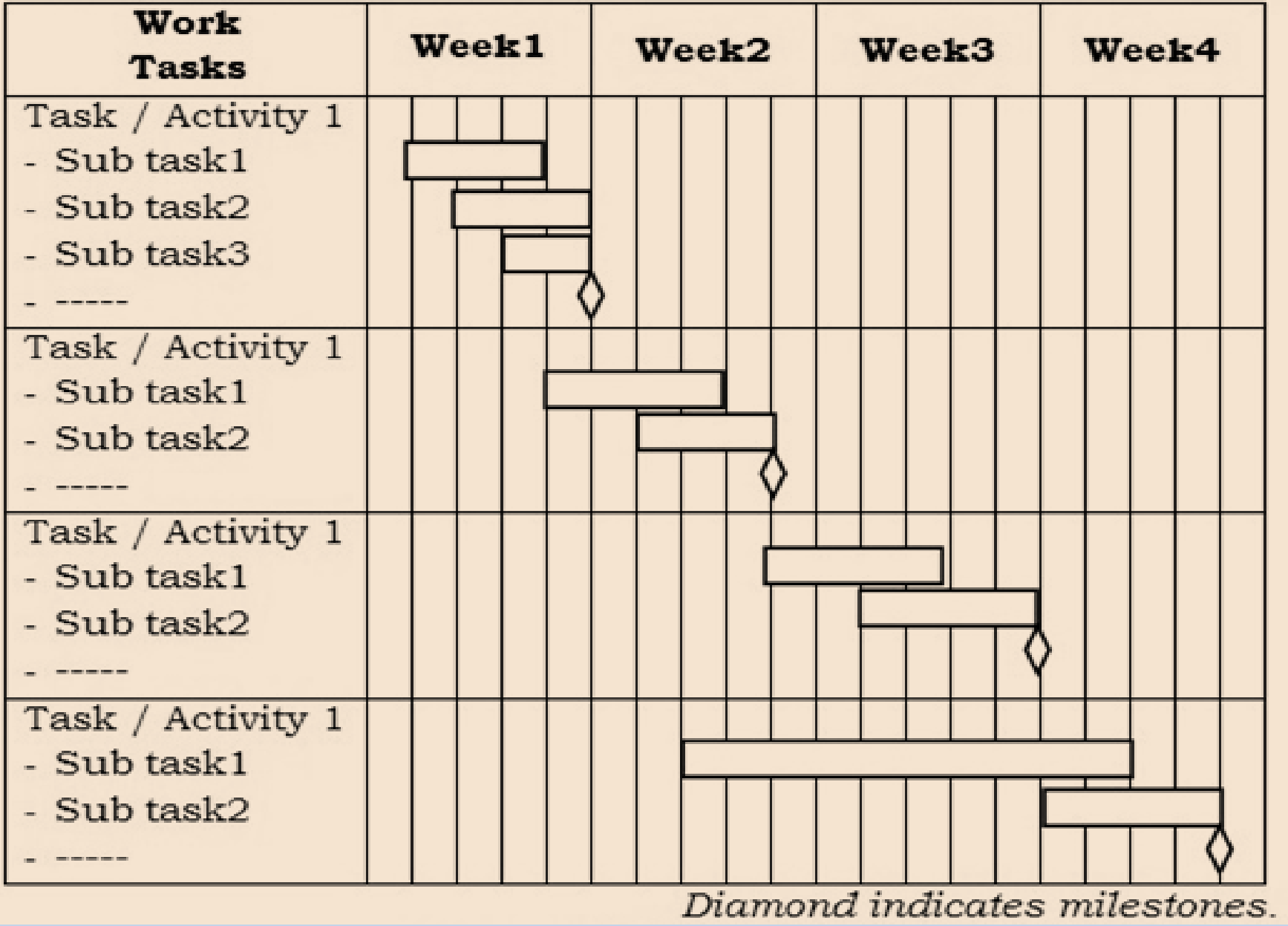
Project Scheduling

- Project scheduling is the process of organizing tasks, resources, and timelines to complete a project within a defined time frame.
- It involves determining what needs to be done, when, and by whom.
- Effective project scheduling ensures tasks are completed in the correct order, within budget, and meets project goals.
- Scheduling Process involves following steps:
  - Define Project Scope and Objectives
  - Identify all the tasks required to complete the project
  - Determine the order in which task should be completed
  - Estimates duration of each task
  - Assigning resources to each task
  - Creating Schedule
  - Optimize the Schedule
  - Monitoring the Progress

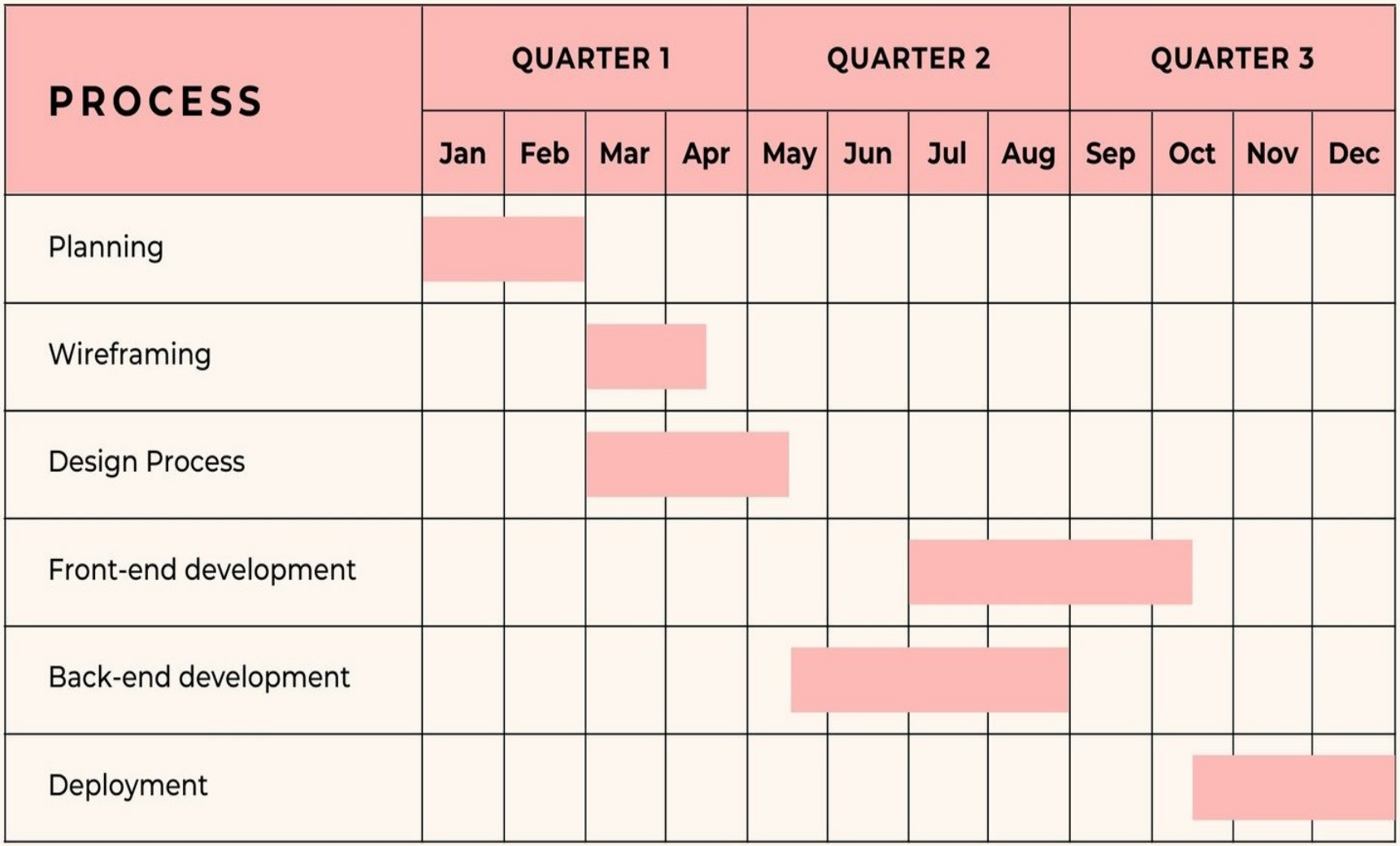
- Benefits of Project Scheduling:
  - Improved Time Management
  - Better Resource Allocation
  - Enhanced Coordination and Communication
  - Increased Accountability
  - Improved Budget Management
  - Facilitates Goal Achievement
  - Adaptability to Change
  - Improved Decision-Making

- **GANTT Chart (Time Line Chart)**
  - Gantt Chart is a visual project management tool used to plan, schedule, and track project progress, It represents tasks on a timeline, making it easier to see when activities start, how long they last, and their dependencies.
  - Gantt chart is a special type of bar chart where each bar represents an activity, Bars are drawn along time line, Length of bar proportional to the duration of time planned.
  - In short, It shows activities against time.
  - Prepared by the Project Manager.
  - How to Plan a Gantt Chart?
    - Identify all the Tasks.
    - If possible, break down the tasks into smaller tasks.
    - Determine the total estimated completion time for each task.
    - Plot activities on GANTT chart.
    - Draw milestones at applicable places.

– Simple Skeleton of GANTT Chart



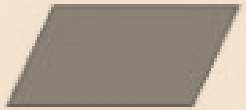
– Example : Website Development Process GANTT Chart



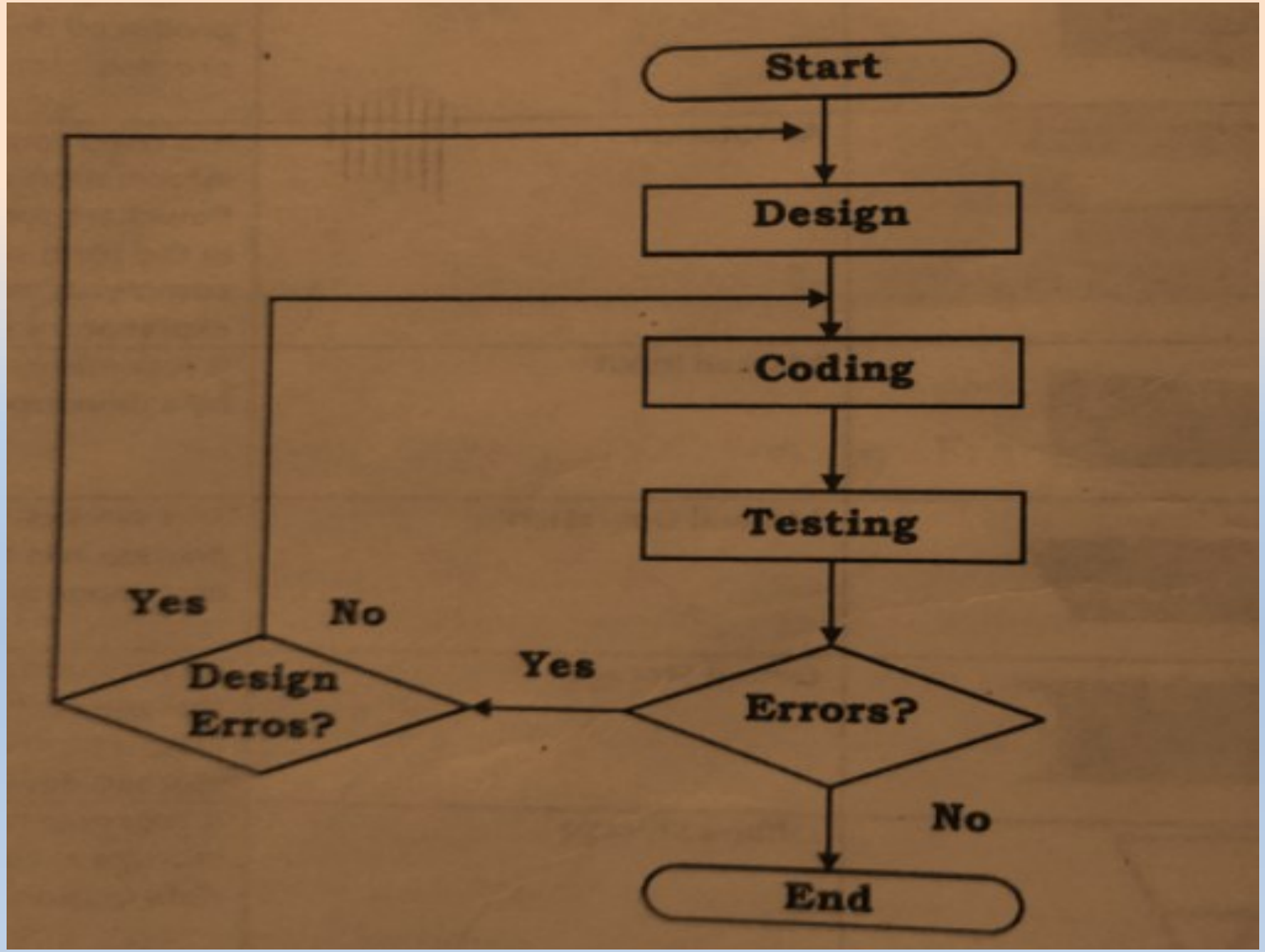
- Advantages:
  - Clear Visual Representation
  - Improved Planning
  - Progress Tracking
  - Resource Management
  - Milestone Management
  - Early Problem Detection
  - Easy to Understand and Use
- Disadvantages:
  - Complexity for Large Projects
  - Limited Flexibility
- Applications:
  - Project Management
  - Software Development
  - Event Planning etc.

- **Flow Chart**
  - It is a graphical representation of a system or a functioning of a system.
  - Flowchart in project scheduling visually represents the sequence of tasks, milestones, and decision points in a project, it is used to map out the workflow and dependencies, making it easier to plan, communicate, and manage project schedules effectively.
  - Flowcharts can be used to display a stage-by-stage progression of project.
  - They effectively show a sequence of events, with the completion of one stage leading to the beginning of another.
  - Steps to Create Flow Chart:
    - Define Scope of Project
    - Identify Task in Sequence
    - Organize Task by Type and Symbol
    - Draw Flowchart

– Symbols Used to Draw a Flow Chart:

Symbol	Symbol Name	Description
	Flow lines	Flow lines are used to connect symbols used in flowchart and indicate direction of flow.
	Terminal (START / STOP)	This is used to represent start and end of the flowchart.
	Input / Output	It represents information which the system reads as input or sends as output.
	Processing	Any process is represented by this symbol. For example, arithmetic operation, data movement.
	Decision	This symbol is used to check any condition or take decision for which there are two answers. Yes (True) or No (False).
	Connector	It is used to connect or join flow lines.

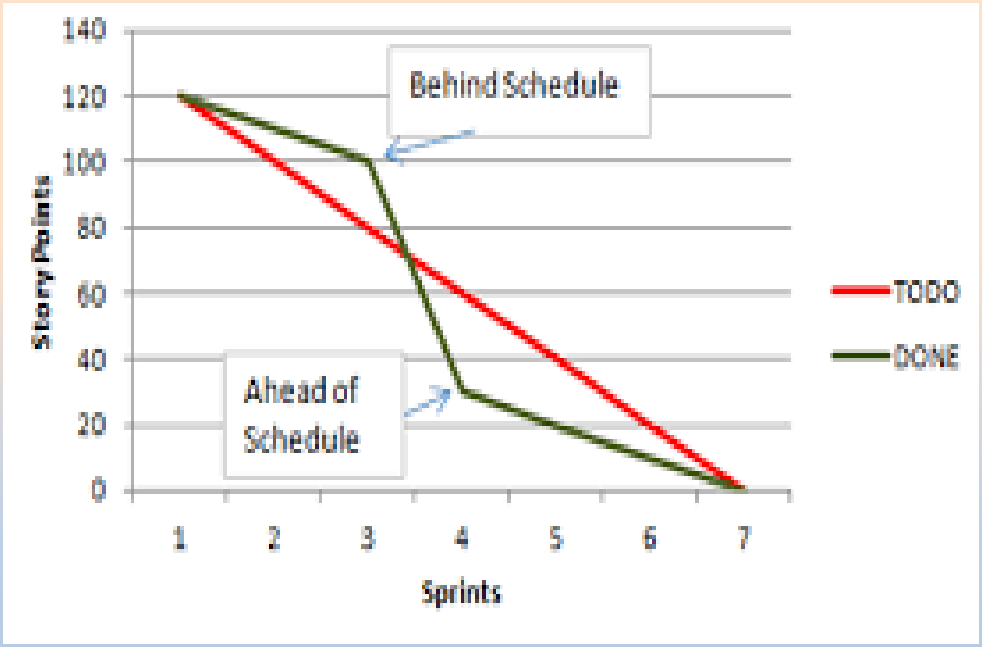
– Example:


Start
Design
Coding
Testing
Errors?
Yes
Design
Erros?
Yes
No
No
End

- Advantages:
  - Clear Visual Representation
  - Easy to Understand
  - Improved Communication
  - Effective Planning
  - Better Resource Management
  - Tracking Progress
- Disadvantages:
  - Complex to Rearrange
  - Time Consuming
  - Complexity for Large Projects
  - Limited Depth
- Applications:
  - Project Management
  - Software Development
  - Event Planning etc.

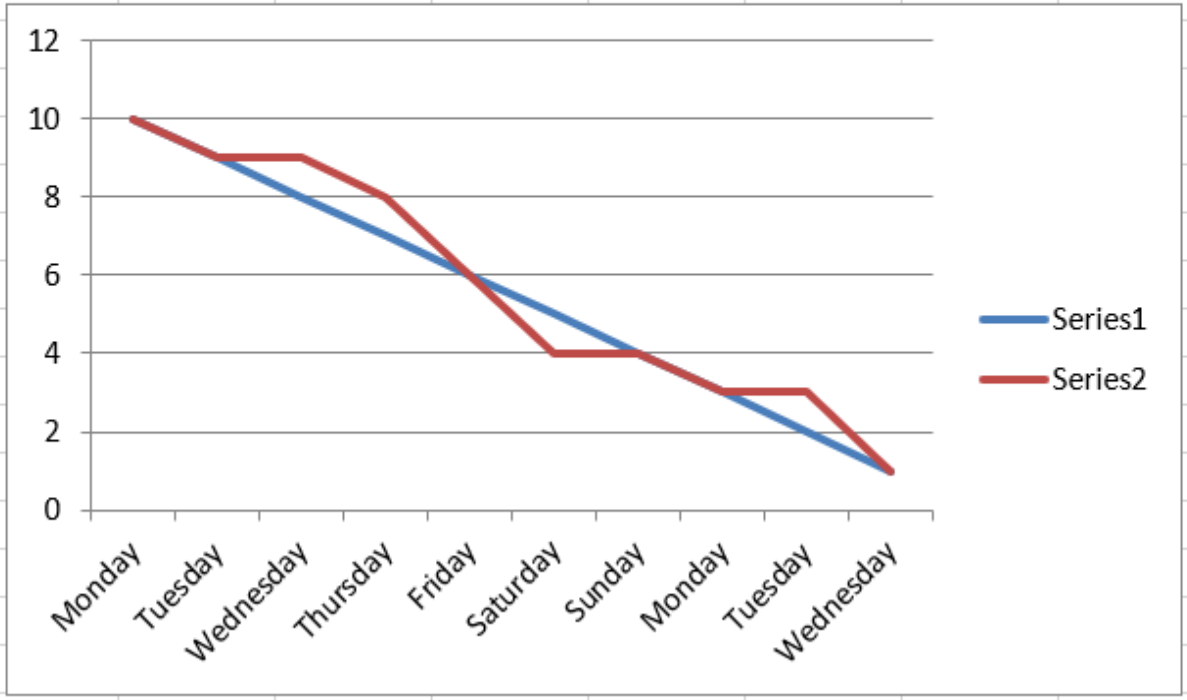
- **Sprint Burndown Chart for Agile Model**
  - In Agile , Sprint is a set period of time during which specific work has to be completed and made ready for the review.
  - Sprint Burn Down Chart is a visual comparison of how much work has been completed during a sprint and the total amount of work remaining.
  - It helps measure a Scrum team's progress, and it provides an easy view of whether the team needs to make any adjustments to complete its work for the current sprint iteration.
  - It is a graph with a y-axis and x-axis, the vertical axis measures the total amount of work that the team estimates it will complete during its current sprint and the horizontal axis shows the number of days remaining until the end of the sprint.
  - On the chart are two lines: the actual work line (a line that represents the team's progress) and the ideal work line (a straight line from the top of the y-axis to the end of the x-axis).
  - This allows the team to track their progress and make adjustments if they are falling behind or ahead the schedule.

– How to Read It: we have two trend lines Ideal (TODO) and Actual (DONE)



– Example:

Sprint Burndown Chart		
Dates	Planned Tasks	Actual Tasks
Monday	10	10
Tuesday	9	9
Wednesday	8	9
Thursday	7	8
Friday	6	6
Saturday	5	4
Sunday	4	4
Monday	3	3
Tuesday	2	3
Wednesday	1	1



- Advantages:
  - Progress Monitoring
  - Early Problem Identification
  - Focus on Deliverables
  - Supports Agile Principles
- Disadvantages:
  - Limited Context
  - Not Suitable for All Teams
- Applications:
  - Agile Project Management
  - Team Motivation

Risk Management

- Software Risk is a problem that could cause some loss or threaten the success of software project, but which hasn't happened yet, this risk may affect negatively to the cost, schedule, technical success or quality of the project.
- Risk Management is the process of identifying, analyzing, and moderating potential risks that can impact the success of a software project.
- It is a crucial part of project management, aiming to reduce the probability of negative outcomes and ensuring that projects are delivered on time, within budget, and with the desired quality.
- Objectives of Risk Management:
  - Identify potential problems and deal with them when they are easier to handle before they become critical.
  - Allow early identification of risks and provide management.
  - Increase the chance of project success.
  - Monitoring and Controlling Risk Prone Activities
  - Ensure Continuity and Stability
  - Improve Resource Utilization

- Different Risk Management Activities are:

- **Risk Assessment**
  - Risk Assessment is the process of identifying, analyzing, and evaluating potential risks that could negatively impact a project, organization, or system.
  - It includes the Following Activities:
  - ***Risk Identification***
    - Risk Identification is the first step in the risk assessment process, it involves systematically recognizing potential threats that could negatively impact a project, organization, or system.
    - The goal is to create a comprehensive list of risks before they become critical issue, in this process some common risks areas are found and checklist is prepared then categorize that risks into different classes.
    - There are three main categories of risks:
      - Project Risks (Poor Planning, Inadequate Resource Allocation, Unrealistic Deadlines)
      - Technical Risks (Unfamiliar Technologies, Complex Integration, Performance Issues)
      - Business Risks (Changing Requirements, Budget Constraints, Market Changes)

- ***Risk Analysis***
  - Risk Analysis is the process of evaluating identified risks to determine their probability, impact, and potential consequences, it helps organizations prioritize risks and develop appropriate moderation strategies.
  - Purpose: To examine how project outcomes might change with modification of risk input variables.
  - The input of this activity is the list of risks developed in risk identification and output is the ranking of the risks.
  - Main Activities of this process are:
    - Group Similar Risks
    - Determine Risk Drivers
    - Determine Source of Risks
    - Estimate Risk Exposure

– ***Risk Prioritization***

- Risk Prioritization is the process of ranking risks based on their potential impact and probability, allowing organizations to focus on the most critical threats first.
- It helps project teams allocate resources and attention to the most critical risks, ensuring efficient management.
- Method of Risk Prioritization
  - Risk Matrix (Probability-Impact Matrix)
  - Risk Scoring (Risk Index Method)
  - Failure Mode and Effects Analysis (FMEA)

- **Risk Control**
  - Risk Control is the process of implementing measures to reduce or eliminate risks that could negatively impact a project, organization, or system.
  - It includes the Following Activities:
  - ***Risk Management Planning***
    - Risk Management Planning is the process of identifying, assessing, prioritizing, and developing strategies to manage potential risks that could impact a project, business, or system
    - It involves identification of strategies to deal with risk, these fall into following categories:
      - Risk Avoidance (Don't do the risky things)
      - Risk Minimization (Risk Reduction to Reduce impact of the risk)
      - Risk Contingency Plan (Deals with risk if it occurs)

- ***Risk Monitoring***
  - Risk Monitoring is the ongoing process of tracking identified risks, identifying new risks, and evaluating the effectiveness of risk mitigation strategies throughout the lifecycle of a project or business operation.
  - Projects can be evaluated periodically to manage the risks.

- ***Risk Resolution***
  - Risk Resolution is the process of addressing and eliminating risks through corrective actions, ensuring they no longer pose a threat to a project, system, or organization.
  - Input of this Activity is : Risk Action Plan
  - Outputs of this Activity are: Risk Status, Acceptable Risks, Reduced Rework, Corrective Actions, Problem Prevention.

GTU Questions

Unit-3 (CO-3)	
Sr No.	**Question**
1	List out responsibilities of Project manager and skills required in project manager.
2	Explain the Line of Code Size Estimation Technique with suitable example. OR What is Metrics for Size Estimation? Explain LOC with example. OR What is Metrics for Size Estimation? Explain FP with example. OR List various components used in calculation of FP. OR Write advantages and disadvantages of LOC
3	Explain the Project Estimation technique using the COCOMO model. OR Explain Project Estimation Techniques using basic COCOMO model OR Explain Project Estimation Techniques using intermediate COCOMO model OR List six phases of complete COCOMO Model.
4	Prepare Gant Chart for any system of your choice. OR Sketch a Gantt chart for the Leave Management System with all SDLC phases completed in 12 weeks. OR Write a short note on Gantt Chart. OR Draw and explain Gantt chart for Online shopping system
5	Sketch Flow chart for a process of vehicle rental system. OR Draw the flowchart for Library Management System. OR Draw the flowchart for CRM (Customer Relationship Management). OR Draw and explain Flow chart for Online shopping system.
6	Prepare Sprint burn down chart for any system of your choice.
7	Explain Risk Assessment in detail. OR Explain Risk identification in detail. OR Define risk. Explain risk identification in risk management. OR Write a short note on Risk Management. OR Explain different types of risks in brief